

MVC, Controllers & Services

Express best practices, clean architecture, and SOLID design

Agenda

1. **Express Best Practices** – keep routes thin
2. **MVC Structure** – separating concerns
3. **Controllers** – handling logic and errors
4. **Services** – data access and business logic
5. **How it follows SOLID** – clean, maintainable code

Part 1

Express Best Practices

The Problem: Logic in Routes

Routes should only **route** – not contain business logic.

```
// ❌ BAD – logic, validation, and data access all in the route
router.get("/posts/:id", async (req, res) => {
  if (!req.params.id || isNaN(Number(req.params.id))) {
    return res.status(400).json({ error: "Invalid ID" });
  }
  const post = mockPosts.find(p => p.id === Number(req.params.id));
  if (!post) {
    return res.status(404).json({ error: "Not found" });
  }
  res.json(post);
});
```

Routes become hard to test, hard to reuse, and hard to maintain.

The Fix: Thin Routes

Routes should **delegate** to a controller – nothing more.

```
// ✅ GOOD – route only maps URL + method to a controller function
import { PostController } from "../controllers/postController";

const controller = new PostController();

router.get("/posts/:id", (req, res) => controller.getById(req, res));
router.get("/posts", (req, res) => controller.getAll(req, res));
router.post("/posts", (req, res) => controller.create(req, res));
router.put("/posts/:id", (req, res) => controller.update(req, res));
router.delete("/posts/:id", (req, res) => controller.delete(req, res));
```

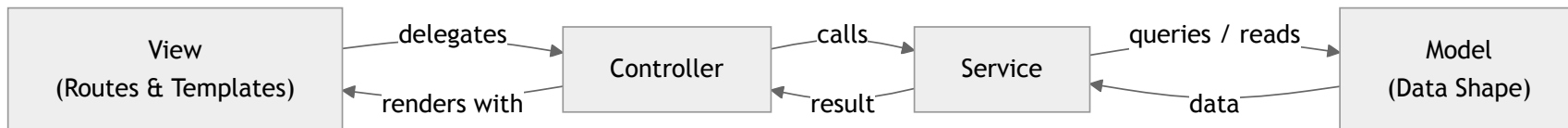
One line per route. No logic. Easy to read at a glance.

Part 2

MVC Structure

What is MVC?

Model-View-Controller separates an application into three layers.



Each layer has **one responsibility** – the foundation of clean architecture.

The Layers Explained

Layer	What it is	Example	For backend
View	User interface & response	HTML templates, JSON response	Routes define what the View returns
Controller	Request handler & orchestrator	<code>PostController</code>	Validates input, calls Service, sends response
Service	Business logic & data access	<code>PostService</code>	Fetches/persists via ORM or mock data
Model	Data structure & validation	<code>post.ts</code>	Defines the shape of your data – covered in Part 4

In this phase, we're building the **backend** – the View will be HTML templates (frontend week).

What is a Promise?

A **Promise** represents a value that isn't available yet – but will be in the future.

```
// Synchronous – blocks until done
const data = readFileSync("data.json"); // 🛑 everything waits

// Asynchronous – returns a Promise, continues immediately
const promise = fetch("/api/posts"); // 🚀 non-blocking
```

State	Meaning
Pending	Operation still in progress
Fulfilled	Completed successfully – value is available
Rejected	Failed – an error occurred

Without Promises, your server would freeze on every database query or API call.

async / await

`async` and `await` let you write asynchronous code that **reads like synchronous code**.

```
// ❌ Without async/await – nested callbacks ("callback hell")
fetch("/api/posts")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// ✅ With async/await – clean and readable
async function getPosts(): Promise<Post[]> {
  const res = await fetch("/api/posts");
  const data = await res.json();
  return data;
}
```

- `async` marks a function as returning a `Promise`
- `await` pauses execution until the Promise resolves
- Errors are caught with `try/catch` – just like synchronous code

You'll see `async / await` and `Promise<T>` throughout our controllers and services.

Part 3

Controllers & Services

Controller Responsibilities

The controller is the **traffic manager** between HTTP and your logic.

- Parse and validate request parameters/body
- Call the appropriate service method
- Handle errors and return correct HTTP status codes
- Never touch the database directly

Rule of thumb: if it knows about `req` or `res`, it belongs in the controller.

A Controller in Practice

```
import { Request, Response } from "express";
import { PostService } from "../services/postService";

export class PostController {
  constructor(private service: PostService = new PostService()) {}

  async getById(req: Request, res: Response): Promise<void> {
    const id = Number(req.params.id);
    if (isNaN(id)) {
      res.status(400).json({ error: "ID must be a number" });
      return;
    }
    const post = await this.service.findById(id);
    if (!post) {
      res.status(404).json({ error: "Post not found" });
      return;
    }
    res.status(200).json(post);
  }
}
```

A Controller in Practice

Why this follows SOLID:

- **Single Responsibility** – controller only handles HTTP, not data access
- **Dependency Inversion** – service is injected, not created inside (swap `MockPostService` for `PrismaPostService` later)
- **Open/Closed** – controller stays closed for modification when you change the database
- **Easy to test** – pass a mock service to test logic without hitting the database

Service Responsibilities

The service handles **data access** and any business logic.

- Fetch or persist data (via an ORM or a mock for now)
- Apply business rules (e.g. calculate totals, enforce limits)
- Return plain data – **no** req / res here
- Easy to swap the underlying data source later (mock → Prisma)

Rule of thumb: if it doesn't know about HTTP, it belongs in the service.

A Service in Practice

```
// src/services/postService.ts

// Mock data – later replaced by a real ORM (e.g. Prisma)
const mockPosts: any[] = [
  { id: 1, title: "Getting Started", content: "TypeScript basics ...", author: "Alice" },
  { id: 2, title: "Advanced Patterns", content: "Generics explained ...", author: "Bob" },
];

export class PostService {
  async findById(id: number): Promise<any> {
    return mockPosts.find(p => p.id === id);
  }

  async findAll(): Promise<any[]> {
    return mockPosts;
  }
}
```

A Service in Practice

Why this follows SOLID:

- **Single Responsibility** – service only handles data access, not HTTP
- **Open/Closed** – swap mock for Prisma without changing the interface
- **Liskov Substitution** – any implementation with the same methods works
- **Testable** – controller tests use mock service; integration tests use real database later

Wiring It All Together

```
// src/routes/postRoutes.ts
import { Router } from "express";
import { PostController } from "../controllers/postController";

const router = Router();
const controller = new PostController();

router.get("/", (req, res) => controller.getAll(req, res));
router.get("/:id", (req, res) => controller.getById(req, res));
router.post("/", (req, res) => controller.create(req, res));
router.put("/:id", (req, res) => controller.update(req, res));
router.delete("/:id", (req, res) => controller.delete(req, res));

export default router;
```

```
// src/app.ts
import postRoutes from "../routes/postRoutes";
app.use("/api/posts", postRoutes);
```

Your File Structure So Far

After adding your controller and service, your project should look like this:

```
src/  
  index.ts                ← entry point – sets up Express and registers routers  
  controllers/  
    postController.ts     ← handles HTTP, validates input, calls service  
  routes/  
    postRouter.ts         ← thin routes, delegates to controller  
  services/  
    postService.ts        ← data access and business logic  
dist/                     ← compiled output (git-ignored)
```

Each folder has **one job** – routes route, controllers orchestrate, services fetch data.

Summary

- **Routes** – thin, one line per endpoint, delegate to controllers
- **Controllers** – handle HTTP, validate inputs, orchestrate service calls
- **Services** – pure data access and business logic, testable independently
- **MVC + SOLID** = clean, testable, maintainable code

Now it's your turn

Exercise 3 – Create `ExpenseController` and `ExpenseService` classes, map expense routes to controller methods, and call the mock service from the controller.